

RASPBERRY PI ON QEMU

1. Download raspbian jessie image :
<https://downloads.raspberrypi.org/raspbian/images/raspbian-2017-04-10/>
2. Unzip .zip file to get raspbian image
3. Download latest qemu kernel: <https://github.com/dhruvvyas90/qemu-rpi-kernel>
4. Create new folder :
`$ mkdir ~/qemu_vms/`
5. Move raspbian image and qemu kernel to ~/qemu_vms/
6. `$ sudo apt-get install qemu-system`
7. `fdisk -l 2017-04-10-raspbian-jessie.img`

```
user@user-Standard-PC-i440FX-PIIX-1996:~/qemu_vms/qemu-rpi-kernel$ fdisk -l 2017-04-10-raspbian-jessie.img
Disk 2017-04-10-raspbian-jessie.img: 3.99 GiB, 4285005824 bytes, 8369152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x402e4a57

Device                                Boot Start    End Sectors  Size Id Type
2017-04-10-raspbian-jessie.img1        8192    92159    83968    41M  c W95 FAT32 (L
2017-04-10-raspbian-jessie.img2       92160 8369151 8276992    3.9G  83 Linux
user@user-Standard-PC-i440FX-PIIX-1996:~/qemu_vms/qemu-rpi-kernel$
```

8. Take the value of filesystem (.img2) 92160 and multiply by 512 = 47185920 bytes. I use this value as an offset in following command

```
user@user-Standard-PC-i440FX-PIIX-1996:~$ sudo mkdir /mnt/raspbian
[sudo: authenticate] Password:
user@user-Standard-PC-i440FX-PIIX-1996:~$ sudo mount -v -o offset=47185920 -t ext4 /home/user/qemu_vms/qemu-rpi-kernel/2017-04-10-raspbian-jessie.img /mnt/raspbian
mount: /dev/loop11 mounted on /mnt/raspbian.
user@user-Standard-PC-i440FX-PIIX-1996:~$
```

9. `sudo nano /mnt/raspbian/etc/ld.so.preload`

Comment out every entry in that file with '#'

```
user@user-Standard-PC-i440FX-PIIX-1996: ~ — sudo n...
user@user-Standard-PC-i440FX-PIIX-1996: ~/c user@user-Standard-PC-i440FX-PIIX-1996: x

GNU nano 8.4 /mnt/raspbian/etc/ld.so.preload *
#//usr/lib/arm-linux-gnueabi/hf/libarmmem.so

[ Read 1 line ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste       ^J Justify    ^_ Go To Line
```

9. \$ cd ~

\$ sudo umount /mnt/raspbian

10. Emulate it on Qemu by using the following command:

```
user@user-Standard-PC-i440FX-PIIX-1996:~$ qemu-system-arm -kernel /home/user/qemu_vms/qemu-rpi-kernel/kernel-qemu-4.4.34-jessie -cpu arm1176 -m 256 -M versatilepb -serial stdio -append "root=/dev/sda2 rootfstype=ext4 rw" -hda /home/user/qemu_vms/qemu-rpi-kernel/2017-04-10-raspbian-jessie.img -nic user,hostfwd=tcp::5022-:22 -no-reboot
WARNING: Image format was not specified for '/home/user/qemu_vms/qemu-rpi-kernel/2017-04-10-raspbian-jessie.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
Uncompressing Linux... done, booting the kernel.
```

I get GUI of raspbian OS

What is RISC vs CISC?

RISC = Reduced Instruction Set Computer

CISC = Complex Instruction Set Computer

RISC uses many simple instructions; CISC uses fewer but more complex instructions.

ARM = RISC. Intel/AMD x86 = CISC.

Simple Instructions (RISC) vs Complex Instructions (CISC)

Example Task: Add two numbers from memory and store the result

Let's say we want to compute: $C = A + B$ where A, B, and C are in memory.

CISC way (x86)

x86 has an instruction called `ADD` that can directly work on memory:

`MOV EAX, [A]` ; load A into register

`ADD EAX, [B]` ; add B (from memory) directly to register

`MOV [C], EAX` ; store result

Notice: x86 instructions can combine multiple operations — load from memory, add, and even modify memory in one instruction. The hardware has to decode complex instructions, which is more work per instruction.

RISC way (ARM)

ARM forces you to break it into small, simple steps:

; ARM assembly (simplified)

LDR R1, [A] ; Load A from memory into register R1

LDR R2, [B] ; Load B from memory into register R2

ADD R3, R1, R2 ; Add R1 + R2, put result in R3

STR R3, [C] ; Store R3 back to memory location C

Every instruction does ONE simple thing:

- LDR = load from memory to register
- ADD = add two registers
- STR = store register to memory

No instruction mixes memory access with arithmetic. This is the Load/Store architecture.

The Load/Store Architecture Explained

This is the most important concept in ARM.

Rule: You CANNOT do arithmetic directly on memory

In ARM, this is illegal:

ADD [C], [A], [B] ; INVALID on ARM

You must go through registers:

LDR R1, [A] ; Step 1: Load A into register

LDR R2, [B] ; Step 2: Load B into register

ADD R3, R1, R2 ; Step 3: Do math in registers

STR R3, [C] ; Step 4: Store result back

A Real C Example

Let's see how C code maps to ARM assembly.

C code:

```
int a = 10;
```

```
int b = 20;
```

```
int c;
```

```
c = a + b;
```

ARM assembly (simplified, Cortex-M):

```
LDR R0, =a      ; Load address of variable 'a'
```

```
LDR R1, [R0]    ; R1 = *a = 10
```

```
LDR R0, =b      ; Load address of variable 'b'
```

```
LDR R2, [R0]    ; R2 = *b = 20
```

```
ADD R3, R1, R2   ; R3 = 10 + 20 = 30
```

```
LDR R0, =c      ; Load address of variable 'c'
```

```
STR R3, [R0]    ; *c = 30
```

Notice the pattern:

1. LDR to get values into registers
2. ADD to compute
3. STR to save back to memory

The compiler does this automatically — you write C, and the compiler produces these load/store sequences.

The x86 Comparison (Side by Side)

Same task: `C = A + B`

x86 (CISC):

asm

```
MOV EAX, [A]      ; load A into register EAX
ADD EAX, [B]      ; add memory B directly to EAX ← memory operand!
MOV [C], EAX      ; store EAX into C
```

3 instructions, but `ADD` here touches memory directly.

ARM (RISC):

asm

```
LDR R1, [A]       ; load A
LDR R2, [B]       ; load B
ADD R3, R1, R2    ; add registers only
STR R3, [C]       ; store result
```

4 instructions, all simple and uniform.

The ARM version has more instructions but each is simpler, faster to decode, and easier to pipeline. Over billions of operations, this translates to better power efficiency.

Other RISC Characteristics

Characteristic	ARM (RISC)	x86 (CISC)
Instruction length	Fixed (16 or 32 bits in Thumb-2)	Variable (1–15 bytes)
Memory access	Only via LDR/STR	Any instruction can access memory
Registers	Many (16 in Cortex-M)	Fewer (8–16 general purpose)
Addressing modes	Few, simple	Many, complex

ARM Data Types

ARM is a 32-bit architecture (in your emulated ARM11). That means most operations happen on 32-bit values. But ARM also handles smaller data.

The Three Sizes

Type	Size	ARM Name	Range
------	------	----------	-------

Byte	8 bits	<code>.byte</code>	0 to 255 (unsigned) / -128 to 127 (signed)
Halfword	16 bits	<code>.halfword</code> / <code>.short</code>	0 to 65,535 / -32,768 to 32,767
Word	32 bits	<code>.word</code> / <code>.long</code>	0 to 4,294,967,295 / -2 ³¹ to 2 ³¹ -1